

Facts, Rules, Inference & Logical Programming

CO528 Lec11 E20

Department of Computer
Engineering

Learning Outcomes

- Define facts and rules
- Explain logical inference
- Differentiate declarative vs procedural programming
- Implement simple logic programs
- Apply reasoning to engineering problems

Why Logical Programming?

- Expert systems reasoning
- AI inference engines
- Policy engines
- Authorization systems

Motivation Example

Medical Diagnosis System:

- fever
- cough
- Rule-based diagnosis

Programming Paradigms

- Imperative / Procedural
- Declarative / Logical

Procedural Programming

- C, Java, Python
- Step-by-step execution
- Control flow based

Declarative Programming

- Focus on what is true
- Define relationships
- System infers solution
- Example: Prolog

What is Logical Programming?

- Based on Predicate Logic
- Knowledge = Facts + Rules
- Computation = Inference

What is a Fact?

- A statement that is unconditionally true
- Represents stored knowledge

Fact Syntax

- `parent(john, mary).`
- `parent(mary, alice).`

Meaning of Facts

- John is parent of Mary
- Mary is parent of Alice

Facts in Engineering Systems

- `service_running(payment_service)`
- `user_authenticated(amal)`
- `node_active(node1)`

Knowledge Base

- Collection of facts
- Represents stored truths

What is a Rule?

- Head :- Body.
- Head is true if Body is true.

Rule Example

- `grandparent(X,Y) :-`
- `parent(X,Z),`
- `parent(Z,Y).`

Understanding Rule Logic

- X is grandparent of Y
- If X parent of Z
- And Z parent of Y

Rules Enable Derivation

- Facts = knowledge
- Rules = derivation
- Enables inference

Engineering Example Rule

- eligible(User) :-
- authenticated(User),
- has_permission(User).

What is Inference?

- Deriving new facts
- Based on facts + rules

Types of Inference

- Forward Chaining
- Backward Chaining

Forward Chaining

- Data-driven
- Start with facts
- Apply rules
- Generate conclusions

Backward Chaining

- Goal-driven
- Start with query
- Work backward

Example Query

?- grandparent(john, alice).

Inference Steps

1. Match rule
2. Substitute variable
3. Check supporting facts
4. Return result

What is Unification?

- Pattern matching
- Match variables with values

Unification Example

parent(john, mary).

parent(john, bob).

?- parent(john, X).

Variable Binding

- Variables: Uppercase
- Atoms: lowercase

Recursive Rule

ancestor(X,Y) :- parent(X,Y).

ancestor(X,Y) :- parent(X,Z), ancestor(Z,Y).

Why Recursion Matters

- Graph traversal
- Dependency resolution
- Service chains

Logical Programming in AI

- Expert systems
- Knowledge representation
- Planning systems

Rule Engines in Cloud Systems

- Policy enforcement
- Access control
- Distributed systems

Access Control Example

- `allow(User) :- authenticated(User), role(User, admin).`

Configuration Management

- Scale if load high
- Failover if node fails

Advantages

- Declarative clarity
- Natural knowledge representation
- Good for reasoning

Limitations

- Performance issues
- Complex inference
- Debugging recursion

Logical vs Microservices

- Declarative reasoning vs distributed services

Mini Activity

- Model university system using facts and rules

Activity Questions

- Is student eligible?
- Multiple prerequisites?
- Extend rule

Critical Thinking

- Large knowledge base?
- Bottlenecks?
- Compare to SQL?

Summary

- Facts
- Rules
- Inference
- Unification
- Recursion
- Applications